WNDRVR
SUPPORT NETWORK

VXWORKS 7 BOOT LOADER USER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Boot Loader User's Guide, SR0650

14 August 2020

1. BOOT LOADER OVERVIEW

A boot loader is an application whose purpose is to load a VxWorks® image onto a target. VxWorks supports several boot loaders and boot strategies. The methods available to you vary, depending on your target architecture.

In general, boot loaders (including third-party and VxWorks boot loaders) can load and boot a VxWorks kernel image from a storage device (USB drive, SD card, on-board flash memory, or hard drive) as well as over a network from an FTP or NFS server. VxWorks can be configured to perform a warm boot, regardless of which boot loader is used. For Intel targets, there are several strategies for warm boots.

Because of the variation in hardware specifications (even among boards of the same architecture), there are many board-specific methods for creating actual boot media (for example, how to create a bootable hard drive, USB flash drive, or how to flash the boot loader). For board-specific information, see the appropriate BSP Supplement Guide for your BSP on the Wind River Support Network page General BSP.

Multiboot Strategy

Intel 32-bit and 64-bit targets can use a multiboot strategy in which a third-party boot loader (GRUB or SYSLINUX) is used to boot a multiboot-compliant boot loader or image. For VxWorks, this can be a full-featured VxWorks kernel or a VxWorks boot loader (provided in ELF format).

For information on booting with GRUB, see About Using GRUB to Boot VxWorks on page 4.

For information on booting with SYSLINUX, see Booting with SYSLINUX on page 7.

Multi-Stage Strategy

Intel 32-bit and 64-bit targets can employ a multi-stage boot strategy where a traditional BIOS or the Unified Extensible Firmware Interface (UEFI) is used to boot a VxWorks boot loader (**bootApp**) or a full-featured VxWorks image (in binary format).

For information on booting with a multi-stage boot loader, see About the Multi-Stage Boot Process on page 8.

For information on booting with the UEFI boot loader, see About the UEFI Boot Loader on page 10.

PXE Boot

Intel 32-bit and 64-bit targets can use a PXE boot strategy, in which PXE loads either a VxWorks boot loader (**bootApp**) or a full-featured VxWorks kernel (in binary format).

For more information, see Booting with PXE on page 12.

U-Boot

For ARM, PowerPC, and RISC-V targets, use the third-party U-Boot boot loader to boot your VxWorks full-featured kernel.

For more information, see About Booting VxWorks with U-Boot on page 14.

VxBL

For ARM and PowerPC targets, use the VxBL stage 1 boot loader to boot your VxWorks full-featured kernel or a VxWorks boot application (**bootApp**).

For more information, see About the VxWorks Boot Loader on page 23.

WNDRVR

Warm Boot

VxWorks has a warm boot facility (**INCLUDE_WARM_BOOT**) that preserves the contents of the persistent memory region (which is used for storing kernel core dump files, error detection and reporting records, and optionally for System Viewer logs). Other warm boot facilities are provided for Intel with multi-stage boot application modes, but they do not preserve the persistent memory region.

WNDRVR

2. BOOT STRATEGY DOCUMENTATION

This document gives an overview of the possible VxWorks target boot strategies available when booting VxWorks.

Every Wind River supplied board support package (BSP) also provides extensive documentation describing the boot strategies supported by that specific BSP.

The Board Support Package (BSP) Readme

The BSP directory will contain at least one readme file. Each readme is formatted in Mardown format. There is one readme file for each target hardware board supported by the BSP. The two tasks of the BSP readme are:

1. to list the driver components supported by this BSP on this board.
2. to provide the most direct approach to booting VxWorks onto this target board.

Note: For an overview of the Markdown plain text format, refer to https://daringfireball.net/projects/markdown/

For example, the **itl_arria10** BSP only supports a single hardware board so the BSP directory contains a single readme file called **itl_arria10_readme.md**. However, the **itl_generic** BSP supports many different families of Intel processor and each family has a different hardware board. The BSP directory therefore contains many readme files, most of which relate to specific processor families (e.g. *cherry_hill_readme.md*), while *itl_generic_readme.md* applies to any other Intel family that is not already documented.

The BSP Supplement Guide

Each Wind River supplied BSP has a BSP Supplement Guide, located in General BSP. The tasks of this document are:

1. to document any additionally supported workflows for booting VxWorks on this target board.
2. to document the boot loaders supported by this BSP and how to use them.
3. to document the configuration of the drivers and kernel components supported by this BSP.

3. BOOTING WITH GRUB

1. About Using GRUB to Boot VxWorks

Depending on your target hardware, you can boot VxWorks using GRUB.

You can load and boot the VxWorks kernel image from a USB storage device, hard drive, or network location. You can only boot a 32-bit boot application image from a bootable USB flash drive or a bootable hard drive.

When working with Intel 32-bit processors, you cannot use GRUB to load **bootApp**-based images for the itl_generic BSP. You can only load boot ROM, **bootrom_uncmp**, and VxWorks images.

Note: You cannot create a VxWorks boot ROM for VxWorks 7. This is a legacy BSP VxWorks build target.

When working with Intel 64-bit processors, GRUB can load either a VxWorks kernel image or a VxWorks boot application. When configuring a 64-bit VxWorks image project (VIP), you must include the GRUB_MULTIBOOT component.

VxWorks provides warm boot facilities for both Intel 32-bit and 64-bit processors. The INCLUDE_WARM_BOOT component is bootloader agnostic and a warm boot will work with any boot loader without requiring the VxWorks boot application facility (**bootApp**) in the middle. For INCLUDE_FAST_REBOOT or INCLUDE_MULTI_STAGE_REBOOT, GRUB must use the **bootApp** as the second-stage boot loader.

VxWorks supports the Multiboot Specification V1. The operating system gets multiboot information from the boot loader, and includes the location of the VxWorks image on the drive and the boot line. The following example shows a typical **menuentry** in the GRUB configuration file (*/your_boot_media* **/boot/grub/grub.cgf**).

```
menuentry "Wind River vxWorks" { multiboot (hd0,msdos1)/images/vxWorks sysbootline:gei(0,0)host:vx
Works h=10.10.0.240 e=10.10.0.41:ffffff00 g=10.10.0.1 u=wruser pw=wruser }
```

For more information on GRUB and the multiboot specification, see the GRUB website and the multiboot specifications.

Related information

Installing GRUB on your Bootable Media on page 4

2. Installing GRUB on your Bootable Media

You can install GRUB and a VxWorks kernel on your bootable media (hard drive, USB flash drive, or an SD card).

Before You Begin

Wind River recommends that you use Ubuntu to create a bootable drive for the target. Ubuntu typically includes **GRUB2** and the fdisk utility required to partition the boot media of the target.

Procedure

1. Create a partition on the bootable media.

   After booting Ubuntu, create a partition using the fdisk utility as a super user as follows:

WNDRVR

1. Find the name assigned to your storage device.

```
$ mount | grep /dev/sd
```

The device should be in the form **/dev/sd***X*, where *X* starts at letter **a** (for example, **/dev/sda**).

2. Start the fdisk utility.

This example assumes that the storage device's name is **/dev/sdb**. Your device is most likely different.

```
$ fdisk /dev/sdb
```

3. Create a single partition of type **W95 FAT32** of 100 Mb or more.

```
Command (m for help): d
Partition number: 1
Command (m for help): n
Partition type:
…
Select (default p): p
Partition number: 1
First sector: ENTER
Last sector, +sectors or +size{K,M,G}…: +100M
Command (m for help): t
Partition number: 1
Hex code: b
Changed system type of partition 1 to b (W95 FAT32)
Command (m for help): w
```

2. Create a new file system on your storage device.

```
$ mkfs.vfat /dev/sdb1
```

3. Install GRUB on the storage device.

```
$ mount /dev/sdb1 /mnt
$ mkdir /mnt/boot
$ grub-install --boot-directory=/mnt/boot --force /dev/sdb1
```

This sets up GRUB in the master boot record in Partition 1, including creation of the **/boot/grub** directory.

Note: The response to the grub-install command may include warnings similar to the following (which are expected):

```
/usr/sbin/grub-bios-setup: warning: File system `fat' doesn't support
embedding.
/usr/sbin/grub-bios-setup: warning: Embedding is not possible. GRUB can
only be installed in this setup by using blocklists. However, blocklists
are UNRELIABLE and their use is discouraged.
Installation finished. No error reported.
```

4. Create a GRUB configuration file with the following file content and save the file:

```
$ vi /mnt/boot/grub/grub.cfg
```

```
set default=0
set timeout=10
set root=(hd0,1)
terminal console
menuentry "Wind River vxWorks 7" { multiboot (hd0,msdos1)/images/vxWorks sysbootline:gei(0,0)ho
st:vxWorks h=10.10.0.240 e=10.10.0.41:ffffff00 g=10.10.0.1 u=wruser pw=wruser
}
```

5. Install your VxWorks 7 kernel image (a fully featured kernel or a **bootApp**) on your bootable storage device.
   1. Plug your storage device into the development host.
   2. Mount it (in this example, it shows as **/dev/sdd1** on the host).

      ```
      $ mount /dev/sdd1 /mnt
      ```

   3. Create the images directory.

      ```
      $ mkdir /mnt/images
      ```

   4. Copy the VxWorks 7 kernel image to the **/mnt/images** directory.

      ```
      $ cp installDir /workspace/default/vxWorks /mnt/images
      ```

      Alternatively, copy a **bootApp** to the **/mnt/images** directory as follows:

      ```
      $ cp installDir /workspace/default_romCompress/vxWorks /mnt/images
      ```

6. Unmount the drive.

   ```
   $ umount /mnt
   ```

Related information

About Using GRUB to Boot VxWorks on page 4

4. BOOTING WITH SYSLINUX

Depending on your target hardware, you can boot VxWorks using SYSLINUX.

You can load and boot the VxWorks kernel image from a USB storage device, hard drive, or network location. You can only boot a 32-bit boot application image from a bootable USB flash drive or a bootable hard drive.

The SYSLINUX boot loader can load a boot ROM or kernel image from a file system.

Note: You cannot create a VxWorks boot ROM for VxWorks 7. This is a legacy BSP VxWorks build target.

Note: When SYSLINUX is used to load the VxWorks image directly, you cannot perform a warm reboot unless you have configured VxWorks with INCLUDE_WARM_BOOT.

For information on using SYSLINUX and creating a configuration to load VxWorks images, see the SYSLINUX website.

WNDRVR

5. BOOTING WITH A MULTI-STAGE BOOT LOADER

1. About the Multi-Stage Boot Process

You can boot a VxWorks kernel image using a multi-stage boot process on Intel Architecture platforms.

Wind River recommends that you boot with a multi-stage process, not with GRUB. A GRUB-loaded VxWorks image does not support warm reboot or fast reboot methods. Therefore, all reboots are a forced cold boot.

VxWorks provides various warm boot facilities. The INCLUDE_WARM_BOOT component is bootloader agnostic and a warm boot will work with any boot loader without requiring the VxWorks boot application facility (**bootApp**) in the middle. To use GRUB with INCLUDE_FAST_REBOOT or INCLUDE_MULTI_STAGE_REBOOT, you must use the **bootApp** as the second-stage boot loader.

Refer to the itl_generic BSP Supplement for a description of how the *multi-stage boot* method functions.

Multi-Stage bootApp Modes

Note: The multi-stage boot modes cannot be used with the INCLUDE_WARM_BOOT component, which provides for a warm reboot that preserves the contents of the persistent memory region (which is used for storing kernel core dump files, error detection and reporting records, and optionally for System Viewer logs). The multi-stage boot facilities do not preserve the contents of the persistent memory region. The default configuration for the itl_generic BSP is INCLUDE_FAST_REBOOT. This component must be removed in order to add the INCLUDE_WARM_BOOT configuration

Table 1. Boot Modes

Name	Description	Required Component
Cold Reboot	Complete reset and re-initialization when a Ctrl-X or reboot command is entered. This mode is used if no warm or fast components are included.	N/A
Warm Reboot	Saves time by only going back to the point where the VxWorks boot starts.	**BUNDLE_MSB_WARM_REBOOT**
Fast Reboot	Saves additional time (over warm reboot) by using a saved copy of the loaded image.	**BUNDLE_MSB_FAST_REBOOT**

Note: In the current release of VxWorks 7, the VxWorks image project (VIP) boot mode bundles are not supported for the itl_generic BSP. The bundles for boot applications, reboot modes, and board bundles are provided for backward compatibility with some VxWorks 7 BSPs (not itl_generic). New development should use the equivalent components (for example, INCLUDE_FAST_REBOOT in place of BUNDLE_MSB_FAST_REBOOT).

For information specific to your target hardware, see the BSP Supplement Guide relating to your BSP.

You can enable fast reboot mode by including the macro BUNDLE_MSB_FAST_REBOOT (which includes the INCLUDE_MULTI_STAGE_BOOT and INCLUDE_FAST_REBOOT components). In fast reboot mode, the boot device is not being accessed. The second stage saves a copy of itself to a memory area at the end of the 0th memory segment (which is pointed to by the descriptor sysPhysRamDesc[0]( )) with a checksum. A warm reboot verifies the checksum then restores this second-stage copy. A fast reboot triggers a warm reboot, which jumps to the original entry point in stage 1 at 0x8010 (**FIRST_TEXT_ADRS** +

ROM_WARM_HIGH). This entry point restores the previously saved second-stage memory and jumps to it. The second-stage image copy uses some RAM.

> Note: If you do not include the INCLUDE_MULTI_STAGE_BOOT component, fast reboot mode is not enabled. Therefore, you can only perform a cold reboot.

If you want to perform a non-fast-reboot, warm reboot (to save RAM space), you can include BUNDLE_MSB_WARM_REBOOT (which includes the INCLUDE_MULTI_STAGE_BOOT and INCLUDE_MULTI_STAGE_WARM_REBOOT components). However, be careful that you do not initialize the boot device in your boot application and VxWorks. A warm reboots jumps back to the original entry point in stage 1 at 0x8010 (**FIRST_TEXT_ADRS** + **ROM_WARM_HIGH**). This entry point then jumps to 16-bit mode and starts everything from scratch; loading the second-stage **bootapp.sys** file again from the boot media. You cannot perform a non-fast-reboot warm reboot with PXE boots. The second-stage image copy uses no RAM.

If VxWorks initialized the device, many BIOS cannot re-access the boot device on a warm reboot. For example, if you use a high-speed USB flash drive as the boot device and you build your boot application or VxWorks image with INCLUDE_EHCI, the BIOS cannot re-access the drive on a warm reboot. As another example, if your boot device is a SATA drive and you build your **bootApp** or VxWorks image with INCLUDE_DRV_STORAGE_AHCI, the BIOS cannot re-access the drive on a warm reboot. The second-stage image copy uses no RAM.

Related information

Use a Multi-Stage Boot Loader With An Intel Target on page 9

2. Use a Multi-Stage Boot Loader With An Intel Target

For detailed instructions on how to build and deploy the multi-stage boot loader on an Intel target board, refer to itl_generic BSP Supplement.

Related information

About the Multi-Stage Boot Process on page 8

WNDRVR

6. BOOTING WITH THE UEFI BOOT LOADER

1. About the UEFI Boot Loader

In addition to (or instead of) the traditional basic I/O services (BIOS) interface, many Intel boards are equipped with unified extensible firmware interface (UEFI). You can use the proprietary Wind River VxWorks UEFI boot loader to boot a VxWorks image.

The Wind River VxWorks UEFI loader ships in source code form. It is a standalone application, not a VxWorks image. The build process creates two UEFI loader application binaries: one for the 32-bit IA-32 architecture, and one for the 64-bit X64 architecture. IA-64 refers to the Itanium platform. The designation X64 refers to the x86_64 platform.

Note: The secure boot features that can be used with the UEFI boot loader are provided by the Security Profile.

You can use the proprietary UEFI boot loader to boot a VxWorks image under the following conditions:

- The Intel board is UEFI-equipped.
- The VxWorks image is a boot image or a fully-featured standalone kernel image with the entry point romInit( ).
- The image is either a binary (**.bin**) file or an ELF file.

Note: UEFI boot loader versions prior to version 1.0.2.0 only support the binary (**.bin**) file format.

Historically, to boot VxWorks on an Intel architecture processor, you needed to use a boot application to load a VxWorks image due to architecture constraints (notably, the fact that Intel architecture platforms would boot in 16-bit mode with only 640 KB of memory available). The romInit( ) code in the boot application switched the CPU to 32-bit protected mode so that VxWorks could run. With the addition of components such as USB and ACPI, the boot application image became too large to fit in 640 KB of memory. VxWorks 6.9 added an intermediate stage to the boot process in the form of the **vxStage1Boot.bin** image. This image is loaded first, and in turn loads the boot application into upper memory.

The UEFI boot process is based on this same strategy, except that the UEFI boot loader takes the place of the **vxStage1Boot.bin** code. The UEFI boot loader loads a VxWorks boot application (**bootApp**) image or a fully featured standalone kernel image.

The two-stage boot process effectively removes the size limitation on boot application images, making room for additional **bootApp** configuration options including:

- Creating a **vxWorks_romCompress.bin** image that implements a typical boot application (for example, a boot ROM to load other VxWorks images).
- Using a **vxWorks_rom.bin** image instead of a compressed boot application.
- Using a fully-featured VxWorks image instead of a boot application image (that is, a **bootApp** built without the PROFILE_BOOTAPP profile).

This means that the system can boot straight to a VxWorks image without the intermediate boot application stage. Therefore, you can decide if you want to retain the boot application stage or not. (The boot application stage can be useful during development.)

Note: Although Wind River fully supports the Wind River UEFI boot loader, you can also use a third party boot loader such as GRUB to boot a VxWorks image.

Related information

How to Use the UEFI Boot Loader On Your Target on page 11
UEFI Boot Loader and Secure Boot on page 11

2. How to Use the UEFI Boot Loader On Your Target

The UEFI boot loader is supported on most Intel target boards. Refer to the **itl_generic** BSP directory and open the readme file that closes relates to your target board. For example, if your board has the Skylake chipset, refer to **skylake_readme.md** in the **itl_generic** BSP directory for instructions on how to setup the UEFI boot loader on your board.

Further details on setting up and using the UEFI boot loader is found in **vxworks-7/pkgs_v2/boot/uefi/README.txt** and the itl_generic BSP Supplement on the Wind River Support Network page General BSP.

Related information

About the UEFI Boot Loader on page 10
UEFI Boot Loader and Secure Boot on page 11

3. UEFI Boot Loader and Secure Boot

A VxWorks system can be created with secure boot capabilities for Intel targets using the UEFI boot loader and firmware.

For further details, refer to the *VxWorks 7 Security Programmer's Guide: Creating a UEFI Boot Loader and VxWorks Image for Secure Boot*.

Related information

About the UEFI Boot Loader on page 10
How to Use the UEFI Boot Loader On Your Target on page 11

7. BOOTING WITH PXE

On Intel 32- and 64-bit processors, you can boot VxWorks using the Pre-boot Execution Environment (PXE) boot loader. PXE is a standard boot method used to boot from a network.

For detailed instructions on using PXE boot to boot an Intel target refer to the Wind River Support Network page General BSP and open the itl_generic BSP Supplement.

Note: To boot using PXE, your target must include a PXE-compatible network interface card (NIC). A PXE-compatible NIC includes a PXE option ROM for BIOS of a PXE boot support.

A diskless target configured with PXE can automatically obtain an IP address and other network configuration information from a Bootstrap Protocol (BOOTP)/DHCP server, along with the IP address and pathname of a TFTP server from which it can fetch a network boot program.

The PXE boot mechanism was originally designed to support systems with a legacy BIOS firmware. It is also supported for newer systems with the UEFI platform firmware. The table below shows some of the differences between the legacy PXE and UEFI PXE implementations.

Table 1. Types of PXE

Types	Requires	Descriptions
Legacy PXE	**vxStage1Boot.bin**	**Boots from a BIOS** **Uses BIOS option ROM** **Loads a 16-bit executable network boot program at 0x7c00**
UEFI PXE	**BOOTX64.EFI**	**Boots from a UEFI firmware** **Uses UEFI network driver option ROM** **Loads a UEFI (BOOTX64.EFI) executable application**

The network boot program obtained from the BOOTP/DHCP server must be different depending on whether the system is using the legacy BIOS PXE or the UEFI PXE boot scheme.

For sytems using a legacy BIOS, the network boot program must be the **vxStage1Boot.bin** bootstrap loader. This boot program transitions the system from 16-bit real mode to 32-bit protected mode and then uses BIOS facilities to load the VxWorks image from the same server. This image can be either a 32-bit of 64-bit VxWorks bootapp, or a VxWorks kernel.

For systems using a UEFI firmware, the network boot program must be the VxWorks UEFI boot loader application, either the **BOOTIA32.EFI** or **BOOTX64.EFI**.

The **BOOTIA32.EFI** image is requried for systems with a 32-bit UEFI builds. Most current systems run a 64-bit UEFI firmware and you must use the **BOOTX64.EFI** image. Note that these are the exact same UEFI loader applications used when booting a VxWorks from local disk storage.

As with the BIOS PXE scheme, once the UEFI firmware has downloaded the VxWorks UEFI boot loader application from the BOOTP/DHCP server, the boot loader application will in turn download the VxWorks image from the BOOTP/DHCP server, which may also be either a 32-bit or 64-bit VxWorks bootapp or VxWorks kernel.

For more information on PXE, see the PXE specification available from Intel:

WNDRVR

http://www.intel.com/design/archives/wfm/downloads/pxespec.htm

8. BOOTING WITH U-BOOT

1. About Booting VxWorks with U-Boot

On ARM and PowerPC systems, you can use the third-party U-Boot boot loader to boot VxWorks.

The U-Boot boot loader is the industry standard for booting ARM and PowerPC boards. Sometimes, the VxWorks BSP contains a subdirectory called **_bootloader** that contains prebuilt U-Boot image files that you must use to boot your target board. Refer to the supplied BSP documentation for instructions on how to deploy the binary files in target board memory.

For example, under the BSP directory **fsl_t2t4** you will find the sub-directory **_bootloader** containing **u-boot_t2080qds.bin**. For instructions on programming **u-boot_t2080qds.bin** into the target board flash memory, refer to *t2080qds_readme.md* in the BSP directory. For guidance on preparing U-Boot compatible VxWorks image files and loading them into target RAM using U-Boot, refer to fsl_t2t4 BSP Supplement.

If your VxWorks BSP does not contain a prebuilt U-Boot image for your board then you must retrieve U-Boot from the vendor and deploy the resulting image file onto your board's memory. For example, the BSP directory **nxp_s32v234** does not contain a prebuilt U-Boot target image file. To prepare the target board with U-Boot, refer to *nxp_s32v234_readme.md* in the BSP directory for deployment details, and also nxp_s32v234 BSP Supplement describes how to use the deployed U-Boot.

Note: In some cases, the target board comes with a vendor supplied, default U-Boot image in board memory that lacks the patches required to enable U-Boot to load VxWorks. In this case, the instructions for replacing the vendor supplied U-Boot image in memory are supplied in the BSP readme file and further details for using U-Boot are supplied in the BSP Supplement document.

U-Boot with VxWorks FIT Images

U-Boot versions 2020.07 and later support loading a VxWorks flattened image tree (FIT) image. A FIT image packages various image types (for example, the kernel and DTB images) into a single uImage that you can load onto your target. For more information on using U-Boot to load a FIT image, see U-Boot and VxWorks FIT Images on page 19.

Related information

Using U-Boot to Boot VxWorks on page 14
Preparing and Booting a ROM-Resident Image (Execute In Place) on page 18
U-Boot and VxWorks FIT Images on page 19
Generating VxWorks FIT Images on page 20

2. Using U-Boot to Boot VxWorks

U-Boot provides different boot options based on the structure of your VxWorks image.

About This Task

Refer to your BSP Supplement Guide and the readme file in the BSP directory for the most direct approach to using U-Boot to boot VxWorks on your target board.

The steps provided below are useful when your BSP does not support **uVxWorks** as a VxWorks image project (VIP) build target. In this case, you must add the U-Boot header to the VxWorks image file and loaded it correctly from the target. The steps below describe this process.

You must have a device tree blob (DTB) file named *your_board* **.dtb** available in your BSP directory when you build the VxWorks kernel image. The DTB file separates the target board device access information from the actual BSP code. Usually, the DTB file can be used either in embedded or independent mode.

Your VxWorks kernel image must be in the default ELF format and be named **vxWorks** .

Procedure

1. Use the **objcopy** tool to convert the VxWorks kernel image from an ELF file to a binary.

   For the ARM architecture, execute the following command:

   ```
   $ objcopyarm -O binary vxWorks vxWorks.bin
   ```

   For the PowerPC architecture, execute the following command:

   ```
   $ objcopyppc -O binary vxWorks vxWorks.bin
   ```

2. Prepend the U-Boot image header to the VxWorks binary.

   VxWorks includes the **mkimage** tool. This tool is used to produce the **vxWorks.uboot** file.

   The **mkimage** tool includes the following options:

   -A

   Sets the architecture to *arch* .

   -O

   Sets the operating system to *os* .

   -T

   Sets the image type to *type* .

   -C

   Sets the compression type *comp* .

   -a

   Sets the load address to *addr* .

   -e

   Sets the entry point to *ep* .

   -n

   Sets the image name to *name* .

   -d

   Uses image data from *datafile* .

   For the ARM architecture, execute the following command:

   ```
   $ mkimage -A arm -O vxworks -T kernel -C none -a 0x80100000 -e 0x80100000 -n vxworks -d vxWorks
   .bin vxWorks.uboot
   ```

For the PowerPC architecture, execute the following command:

```
$ mkimage -A ppc -O vxworks -T kernel -C none -a 0x00100000 -e 0x00100000 -n vxworks -d vxWorks
.bin vxWorks.uboot
```

The load address and entry point address are target board specific. The load address is the address to which U-Boot copies the VxWorks kernel image (that is, the kernel link address). The entry point is the address of the first instruction to execute. U-Boot jumps to this entry point address after it completes the preparation tasks. These two addresses are usually the same in VxWorks. The BSP uses the address **RAM_LOW_ADRS**, because VxWorks always has its entry point at the start of the image. Refer to the BSP readme file and BSP Supplement Guide for further address details for your target board.

3. Execute the following command to embed the DTB file into the VxWorks binary using the **wrtool** facility:

```
$ prj build -target vxWorks.bin
```

4. Repeat step 2 on page 15 to prepend the **vxWorks.bin** image with the U-Boot image header.

   Alternatively, use a compressed VxWorks kernel image.

   U-Boot can boot a compressed VxWorks kernel image that uses the gzip, bzip2, lma, or lzo compression format. You can compress a VxWorks binary with or without an embedded DTB.

   Execute the following command to compress the image:

```
$ gzip --best vxWorks.bin
```

   The resulting file is **vxWorks.bin.gz**.

   Now prepend the U-Boot image header to the **vxWorks.bin.gz** as follows:

```
$ mkimage -A ppc -O vxworks -T kernel -C gzip -a 0x00100000 -e 0x00100000 -n vxworks -d vxWorks
.bin.gz vxWorks.uboot
```

5. Power on the target and wait for the U-Boot prompt (**=>**).
6. Execute the standard setenv command to provide the boot line.

   Note: VxWorks 7 uses the standard boot line.

```
=> setenv bootargs cpsw(0,0).....
```

   The DTB stores the boot line passed to the VxWorks kernel. Specifically, the boot line is written into the chosen node of the device tree, as required by the ePAPR standard. The VxWorks 7 kernel extracts this information from the device tree and parses it as needed.

   U-Boot provides the boot line for the kernel. If there is no chosen node, U-Boot creates one in the device tree, then updates the boot line. If the environment variable bootargs is set, U-Boot uses it to update the **bootline** property under the chosen node. If the **bootline** property already exists, U-Boot overrides it. Regardless of whether or not there is a valid boot line, U-Boot creates the chosen node.

   Note: With an embedded DTB, U-Boot cannot update the boot line inside the DTB. Therefore, you cannot pass the variable bootargs. In this case, the VxWorks kernel only uses the boot line that is hard-coded into the device tree.

7. Boot VxWorks according to your system requirements.

When using a separate DTB, nodes such as memory, bootargs, enable-method, and cpu-release-addr can be updated by U-Boot. Check the BSP Supplement Guide for information and suitable values for these nodes.

Note: With an embedded DTB, the nodes cannot be accessed by U-Boot.

In some cases, you must set additional environment variables. That is, variables that are passed by U-Boot to the VxWorks kernel image or variables used by U-Boot. This is done from the U-Boot command prompt using the setenv and the saveenv commands. Typical environment variables you may need to set are:

bootargs

This is the boot line used by the kernel for example:

```
"motetsec(2,0)host:vxWorks h=10.1.0.240 e=10.1.0.41:fffffe00 g=10.1.0.1 u=vxworks pw=vxwork
s f=0x0"
```

fdt_high

The DTB is relocated to be below this address. The relocated DTB address must be accessible for the VxWorks kernel image.

bootm

Copies the kernel to the kernel load address defined in **uImage**. U-Boot passes the relocated address of the DTB in a register to the kernel.

Option	Description

Boot VxWorks with a separate DTB from a storage device.	First, load the **kernel** and **dts** files from the storage media, then use the bootm command with the parameters kernel address - dtb address. For example, execute the following commands to load and run a VxWorks kernel on an ARM board from an SD card.
	`=> mmc rescan`
	`=> fatload mmc 0 82000000 vxWorks.uboot`
	`=> fatload mmc 0 80005000 am335x.dtb`
	`=> bootm 0x82000000 - 0x80005000`

Boot VxWorks with a separate DTB over the network.	U-Boot provides a method for TFTP or NFS clients to load both files over the network.
	Execute the following commands to set up the U-Boot networking parameters:
	`=> setenv ethaddr 00:04:9f:ef:01:01`
	`=> setenv ipaddr 192.168.10.5`
	`=> setenv netmask 255.255.255.0`
	`=> setenv serverip 192.168.10.2`
	`=> setenv gatewayip 192.168.10.1`
	`=> saveenv`
	Execute the following command to download the files with a separate DTB:
	`=> tftp 80300000 uVxWorks`
	`=> tftp 80e00000 your-board.dtb`

Option	Description
	Execute the bootm command to boot the VxWorks image with the separate DTB file. `=> bootm 0x80300000 - 0x80e00000`

Boot VxWorks with an embedded DTB.	First, load the **kernel** file from the storage media, then use the bootm command with the parameter kernel address.
	Note: U-Boot will not fix any problems for the kernel because it cannot know the location of the embedded device tree. Even if U-Boot had this information, fixing a device tree could require resizing the blob, which is not practical for embedded mode.
	Execute the following commands to load and run a VxWorks kernel with an embedded device tree on an ARM board from an SD card:
	`=> mmc rescan`
	`=> fatload mmc 0 82000000 vxWorks_dtb.uboot`
	`=> bootm 0x82000000`

Boot VxWorks using the legacy go command with the kernel entry address parameter.	Execute the following command to boot a VxWorks kernel binary with an embedded DTB: `=> go 0x82000000`

Related information

About Booting VxWorks with U-Boot on page 14
Preparing and Booting a ROM-Resident Image (Execute In Place) on page 18
U-Boot and VxWorks FIT Images on page 19
Generating VxWorks FIT Images on page 20

3. Preparing and Booting a ROM-Resident Image (Execute In Place)

VxWorks supports ROM-resident images that execute directly from memory devices such as NOR flash.

About This Task

If your BSP supports booting a ROM-resident VxWorks image from U-Boot, this will be documented in the BSP readme and BSP Supplement Guide documents. Refer to these documents for details.

Alternatively, the following steps may be useful if this feature is unsupported on your BSP. If you choose this method, you must perform additional work. For example, there must be a 64-byte gap between the load address and the entry point, all filled with **0xff**, because the standard U-Boot header is 64 bytes.

Procedure

1. Modify the BSP file **sysALib.s** to reserve space for the 64-byte U-Boot header.

   For example:

```
FUNC_LABEL(_sysInit)
FUNC_BEGIN(sysInit)
#ifdef UBOOT_XIP
.fill 16,4,0xff /* reserve for XIP header */
#endif
```

2. Execute the mkimage command on the workstation with the -x option to add a U-Boot header that supports XIP (a special header).

   For example:

```
$ mkimage -x -A arm -O vxworks -T kernel -C none -a 0x80100000 -e 0x80100000 -n vxworks -d vxWo
rks.bin vxWorks.uboot
```

   Note: You must use an embedded device tree for ROM-resident images.

   Note: Refer to your BSP for target specific memory addresses.

3. Execute the bootm command from the target U-Boot console to boot the ROM-resident image.

   For example:

```
=> bootm 0x80100000
```

   Note: Use the address you included as a parameter to the mkimage command.

Related information

About Booting VxWorks with U-Boot on page 14
Using U-Boot to Boot VxWorks on page 14
U-Boot and VxWorks FIT Images on page 19
Generating VxWorks FIT Images on page 20

4. U-Boot and VxWorks FIT Images

U-Boot versions 2020.07 and later support loading a VxWorks flattened image tree (FIT) image created with the mkimage utility and the appropriate ***.its** definition file.

A FIT image packages various image types (for example, the kernel and DTB images) into a single uImage that you can load onto your target.

Using FIT images allows for image integrity protection using sha1 and md5 checksums.

Refer to the relevant BSP supplement guide on the Wind River Support Network to see if this image type is supported for your BSP and for a description of how to use it. You must also confirm that the BSP supports standard device tree blob (DTB) format.

The BSP may support automatic FIT image generation, or you may have to manually generate a FIT image; see Generating VxWorks FIT Images on page 20.

To use FIT images, set the following in the U-Boot bootargs environment.

```
f=0x1
```

Load the single binary with the following command.

=> **bootm** *address*

For more information, see https://github.com/u-boot/u-boot/blob/master/doc/uImage.FIT/howto.txt.

Related information

About Booting VxWorks with U-Boot on page 14
Using U-Boot to Boot VxWorks on page 14
Preparing and Booting a ROM-Resident Image (Execute In Place) on page 18
Generating VxWorks FIT Images on page 20

5. Generating VxWorks FIT Images

If the BSP that you are using includes an example **.its** definition file, a VxWorks FIT uImage will be generated automatically when you build an associated VxWorks image project (VIP); however, you can also configure and generate a FIT image manually.

About This Task

You can refer to the relevant BSP supplement guide on the Wind River Support Network to see if the VIP build process automatically generates a FIT image.

If the FIT image is not generated automatically, or if you want to use a customized **.its** file, perform the following steps to manually generate a FIT image.

Procedure

1. Configure an **.its** file.

   You must include a reference to the location of the VxWorks kernel and DTB image files.

   The following is an example **.its** file.

```
/*
 * Simple U-Boot uImage source file containing a single kernel and FDT blob
 */

/dts-v1/;
```

```
/ {
        description = "Simple image with single VxWorks and FDT blob";
        #address-cells = <1>;

        images {
                kernel {
                        description = "ls1028 VxWorks kernel";
                        data = /incbin/("./vxWorks.bin.gz");
                        type = "kernel";
                        arch = "arm64";
                        os = "vxworks";
                        compression = "gzip";
                        load = <0x80100000>;
                        entry = <0x80100000>;
                        hash-1 {
                                algo = "crc32";
                        };
                        hash-2 {
                                algo = "sha1";
                        };
                };
                fdt {
                        description = "Flattened Device Tree blob";
                        data = /incbin/("./ls1028a-rdb-pa.dtb");
                        type = "flat_dt";
                        arch = "arm64";
                        compression = "none";
                        load = <0x90000000>;
                        hash-1 {
                                algo = "crc32";
                        };
                        hash-2 {
                                algo = "sha1";
                        };
                };
        };

        configurations {
                default = "conf-1";
                conf-1 {
                        description = "Boot VxWorks kernel with FDT blob";
                        kernel = "kernel";
                        fdt = "fdt";
                };
        };
};
```

2. Copy the kernel and DTB image files to the referenced location.
3. Enter VxWorks shell environment and create the FIT (**.itb**) image.

```
$ mkimage -f itsName .its fitName .itb
```

Results

You can now load the single FIT uImage onto your target.

Related information

About Booting VxWorks with U-Boot on page 14
Using U-Boot to Boot VxWorks on page 14
Preparing and Booting a ROM-Resident Image (Execute In Place) on page 18
U-Boot and VxWorks FIT Images on page 19

9. BOOTING WITH VXBL

1. About the VxWorks Boot Loader

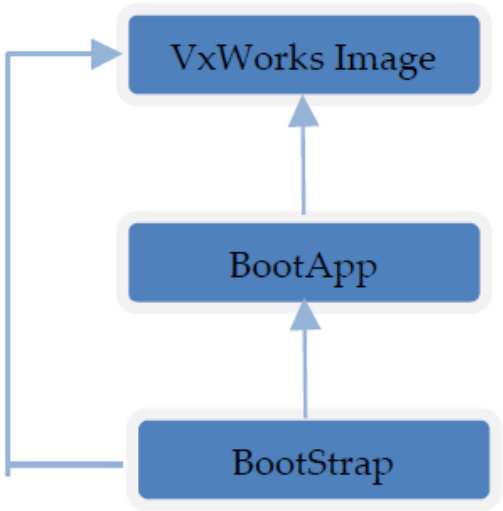
The VxWorks boot loader (VxBL) *bootstrap* (a first stage boot loader) is provided and fully supported by Wind River for both ARM and PowerPC targets. VxBL can boot a full-featured VxWorks kernel or a VxWorks boot application (**bootApp**).

To ensure proper target hardware initialization, Wind River recommends the use of a vendor-provided U-Boot boot loader. As an alternative to U-Boot, you can use VxBL as specified in the *Target Boot Workflows* chapter of the BSP supplement for your board. This can be found in the General BSP category located on the Wind River product documentation site.

If VxBL is not supported for your board, contact Wind River Professional Services.

VxBL High-Level Functionality

The overall run-time architecture of the VxWorks boot loader process is shown below, starting with CPU reset (shown at the bottom) and working upwards.



Two boot sequences are supported as follows:

- The bootstrap (or the first stage boot loader) loads a VxWorks-based boot application. This image is used to load a VxWorks kernel image that you configure to support the applications running on your device.
- The bootstrap loads and runs the fully-featured VxWorks kernel image that you configure to support the applications running directly on the device.

Bootstrap Responsibilities

The bootstrap is the first piece of software that a CPU executes after reset. It is responsible for the following tasks:

- Processor- and SoC-dependent initialization.
- Board-dependent initialization.

- Loading the next image from simple memory-based devices, including NOR, NAND, I2C/EEPROM, SPI FLASH, or SD/MMC (limited FAT32 reader).
- Optionally, loading a flattened device tree (FDT) image into memory with possible run-time modification based on dynamically discovered resources (for example, RAM size or exact CPU type).
- Transferring control to the next image, which is a full-featured VxWorks kernel or the VxWorks-based boot application.

The bootstrap image is a separately built *bare-metal* application. It does not use the VxWorks operating system. This results in a small image that can fit into most constrained environments.

Boot Partitions

The boot partition configuration serves as the VxBL file system. The partitions are as follows:

- uVxWorks: Stores the uVxWorks image.
- vxWorks: Stores the VxWorks kernel image.
- NVRAM: Stores environment variables.
- vxboot: Reserved for the VxBL image itself.

When a partition is not needed for a given scenario, for example the vxWorks partition, you can remove the partition and increase the size of the uVxWorks partition. Use the **uVxWorks flash partition size** (**UIMAGE_PART_SIZE**) BSP parameter of the make config command to change the default size of the uVxWorks partition. For more information, refer to the *Configuration and Setup* chapter of the BSP supplement for your board.

The default and maximum uVxWorks partition sizes for PPC targets that support VxBL are as follows:

Target	Default uVxWorks Partition Size	Maximum uVxWorks Partition Size
fsl_p1p2	0x5c0000 (6,029,312 bytes)	0x5c0000 (6,029,312 bytes)
fsl_p3p4p5 fsl_t1 fsl_t2t4	0x800000 (8,388,608 bytes)	0xc00000 (12,582,912 bytes)

Note: If the uVxWorks image size exceeds the specified uVxWorks partition size, the target will not boot.

Related information

About Building and Using VxBL to Load VxWorks on page 24
Booting a Compressed VxWorks Kernel Image with a Separate DTB on page 25
Booting a VxWorks Boot Application on page 26
Booting a VxWorks Kernel Image as a Payload of the VxBL on page 29
Overwriting the Boot Arguments in the DTB from the VxBL on page 29

2. About Building and Using VxBL to Load VxWorks

Instructions for building and deploying VxBL on your target board are located in the BSP Supplement. To find this, navigate to the General BSP category located on the Wind River product documentation site.

Related information

About the VxWorks Boot Loader on page 23
Booting a Compressed VxWorks Kernel Image with a Separate DTB on page 25

Booting a VxWorks Boot Application on page 26
Booting a VxWorks Kernel Image as a Payload of the VxBL on page 29
Overwriting the Boot Arguments in the DTB from the VxBL on page 29

3. Booting a Compressed VxWorks Kernel Image with a Separate DTB

The VxWorks VxBL boot loader can be used to boot a compressed VxWorks kernel image with a separate device tree blob (DTB).

Before You Begin

This procedure is not normally documented in the BSP Supplement Guide. However, it assumes that you have followed the instructions in the BSP Supplement Guide to configure and build a VxBL to boot from a compressed file and have installed it on a properly formatted bootable media compatible with your target so that, when you power on your board, the VxBL executes.

Procedure

1. Convert your vxWorks image to a **vxWorks.bin** image, compress it (**.gz** format), and add the U-Boot header.

   Use the following command:

```
$ objcopyppc -O binary vxWorks vxWorks.bin gzip --best vxWorks.bin mkimage -O vxworks -T kerne
l -C gzip -n vxworks -A ppc -a 0x100000 -e 0x100000 vxWorks.bin.gz uVxWorks
```

   Note: The addresses used in this command are board specific.

2. Load and boot the compressed kernel with the U-Boot header (uVxWorks).

```
VxWorks fsl_p1p2 Stage 1 Bootstrap (32-bit version)
Built on: Dec 12 2014 14:38:55

mounting flash OK
info - loader VxWorks 7 registered

vxbl#
vxbl#env unset bootargs
vxbl#boot 0x3000000 - 0xf000000
uncompressing image to 0x100000 from 0x3000040, size = 0xeaf1a bytes
starting VxWorks7 @0x100000 dtb@0xf000000...
Hello, VxWorks!
check system clock frequency
system clock frequency: 100000000
...
...
Target Name: vxTarget


 ________           ________
 \77777777\         /77777777/
  \77777777\       /77777777/
   \77777777\     /77777777/
    \77777777\   /77777777/
     \77777777\  \7777777/
```

```
      \77777777\   \77777/              VxWorks 7
       \77777777\   \777/
        \77777777\   \7/     Core Kernel version: 1.0.6.0
         \77777777\   -      Build date: Jan 14 2015 09:47:15
          \77777777\
           \7777777/         Copyright Wind River Systems, Inc.
            \77777/   -                 1984-2014
             \777/   /7\
              \7/   /777\
               -   -------

             Board: Freescale P2020RDB
     OS Memory Size: 512MB
   ED&R Policy Mode: Deployed


 Adding 8190 symbols for standalone.

->
```

Related information

About the VxWorks Boot Loader on page 23
About Building and Using VxBL to Load VxWorks on page 24
Booting a VxWorks Boot Application on page 26
Booting a VxWorks Kernel Image as a Payload of the VxBL on page 29
Overwriting the Boot Arguments in the DTB from the VxBL on page 29

4. Booting a VxWorks Boot Application

You can use VxBL to load the boot application rather than the VxWorks kernel image file.

About This Task

This procedure is not normally documented in the BSP Supplement Guide, but is similar to the procedure for loading a standard VIP image from VxBL.

Creation of a VxWorks **bootApp** image is similar to the creation of a VxWorks kernel image (VIP), except you must also specify the VIP profile, PROFILE_BOOTAPP, when selecting the profile for the VIP. The VSB upon which you base a VIP with the **PROFILE_BOOTAPP** profile must be set to uniprocessor mode (which is not the default for VSB projects). For wrtool, use the -up option. For Workbench, select the uniprocessor option from the Processor Mode drop-down list in the Project Setup dialog.

The VxWorks boot application can boot a fully featured VxWorks kernel from the network (using TFTP) or from a local file system (such as an SD card). To do so, you must explicitly add the network or file system components when configuring the **bootApp** project.

To use VxBL to load and boot the VxWorks boot application (second stage bootstrap), proceed as follows:

Procedure

1. Place the VxWorks boot application (built in **uVxWorks** format) on the SD card.
2. Load the boot application and DTB.

```
vxbl@ load /fat0/bootapp 0x3000000
vxbl@ load /fat0/p2020rdb.dtb 0xf000000
```

Note: All addresses are target board specific. Refer to the BSP Supplement for further details.

3. Boot the VxWorks boot application.

```
vxbl@ boot 0x3000000 - 0xf000000
```

```
Copyright 2012-2014 Wind River Systems

VxWorks fsl_p1p2 Stage 1 Bootstrap (32-bit version)
Built on: Dec 12 2014 14:38:55

mounting FAT O
K
info - loader VxWorks 7 registered

vxbl@ env unset bootargs
vxbl@ load /fat0/bootapp 0x3000000
vxbl@ load /fat0/p2020rdb.dtb 0xf000000
vxbl@ boot 0x3000000 - 0xf000000
copying image to 0x1000000 from 0x3000040, size = 0xef380 bytes
starting VxWorks7 @0x1000000 dtb@0xf000000...
Target Name: vxTarget



Press any key to stop auto-boot...

 7

[VxWorks Boot]: p

boot device          : motetsec
unit number          : 0
processor number     : 0
host name            : host
file name            : vxWorks
inet on ethernet (e) : 10.10.0.2:fffffe00
host inet (h)        : 10.10.0.240
gateway inet (g)     : 10.10.0.1
user (u)             : wruser
ftp password (pw)    : wruser
flags (f)            : 0x80

[VxWorks Boot]:
```

In this case, the boot application loads and boots the fully featured VxWorks kernel from the host's TFTP server.

```
[VxWorks Boot]: @

boot device          : motetsec
unit number          : 0
processor number     : 0
```

```
host name            : host
file name            : vxWorks
inet on ethernet (e) : 10.10.0.2:fffffe00
host inet (h)        : 10.10.0.240
gateway inet (g)     : 10.10.0.1
user (u)             : wruser
ftp password (pw)    : wruser
flags (f)            : 0x80

Loading... 2012328 + 232 + 680 + 286008
Starting at 0x100000...

Hello, VxWorks!
check system clock frequency
system clock frequency: 100000000
obtain CPU information
        CPU 0:
                i-cache-size:           32768
                d-cache-size:           32768
...
...
Target Name: vxTarget


 _________           _________
 \77777777\          /77777777/
  \77777777\        /77777777/
   \77777777\      /77777777/
    \77777777\    /77777777/
     \77777777\   \777777/
      \77777777\   \77777/              VxWorks 7
       \77777777\   \777/
        \77777777\   \7/     Core Kernel version: 1.0.6.0
         \77777777\   -      Build date: Dec 10 2014 15:08:33
          \77777777\
           \7777777/         Copyright Wind River Systems, Inc.
            \77777/   -                 1984-2014
             \777/   /7\
              \7/   /777\
               -   -------

              Board: Freescale P2020RDB
     OS Memory Size: 512MB
   ED&R Policy Mode: Deployed


 Adding 8190 symbols for standalone.

->
```

Related information

About the VxWorks Boot Loader on page 23
About Building and Using VxBL to Load VxWorks on page 24
Booting a Compressed VxWorks Kernel Image with a Separate DTB on page 25
Booting a VxWorks Kernel Image as a Payload of the VxBL on page 29
Overwriting the Boot Arguments in the DTB from the VxBL on page 29

5. Booting a VxWorks Kernel Image as a Payload of the VxBL

The VxWorks VxBL boot loader is used to boot a VxWorks kernel image that is a payload of the VxBL.

Before You Begin

This procedure assumes that you have successfully configured and built a VxBL to boot a VxWorks kernel image that is a payload of the VxBL and installed it on a properly formatted bootable media compatible with your target so that, when you power on your board, the VxBL executes.

Procedure

Once VxBL boots, use the bootp command to load the payload kernel.

Related information

About the VxWorks Boot Loader on page 23
About Building and Using VxBL to Load VxWorks on page 24
Booting a Compressed VxWorks Kernel Image with a Separate DTB on page 25
Booting a VxWorks Boot Application on page 26
Overwriting the Boot Arguments in the DTB from the VxBL on page 29

6. Overwriting the Boot Arguments in the DTB from the VxBL

From the VxBL you can display the boot argument environment variables, modify them, and save the changes to a device tree blob (DTB) that is separate from VxWorks.

Procedure

1. Display the current boot argument environment variables.

```
vxbl@ env print
```

2. Set or modify the boot arguments.

   Use the following syntax:

```
vxbl@ env set bootargs your_bootline
```

   For example:

```
vxbl@ env set bootargs "motetsec(0,0)host:vxWorks h=10.10.0.240
e=10.10.0.2:fffffe00 g=10.10.0.1 u=wruser pw=wruser f=0x80"
```

3. Save the changes.

```
vxbl@ env save
```

Related information

About the VxWorks Boot Loader on page 23

About Building and Using VxBL to Load VxWorks on page 24
Booting a Compressed VxWorks Kernel Image with a Separate DTB on page 25
Booting a VxWorks Boot Application on page 26
Booting a VxWorks Kernel Image as a Payload of the VxBL on page 29

10. VXWORKS BOOT APPLICATION

1. VxWorks Boot Application

The VxWorks boot application is a second-stage boot loader, which is sometimes referred to as the bootApp or BOOTAPP. It can be used to boot both uniprocessor and SMP VxWorks images.

The boot application can be booted by GRUB, SYSLINUX, Bios/UEFI, PXE, U-Boot, or VxBL instead of a VxWorks operating system image. Then, the boot application can boot VxWorks. The boot application is a RAM-based, and cannot be used as a ROM-based boot loader (that is, in a non-volatile device such as flash memory or EEPROM) with VxWorks 7.

Instructions for building and deploying the VxWorks boot application on your target board are located in the BSP Supplement. To find this, navigate to the General BSP category located on the Wind River product documentation site.

The VxWorks boot application has several advantages, particularly in the development environment. It includes a full network stack, and can use FTP or RSH to load VxWorks (instead, for example TFTP, which U-Boot uses). The boot application command shell allows for easy modification of boot parameters, such as the location of the VxWorks image.

During the boot process, the target uses its serial port to communicate with the host system. The boot loader displays a banner page and then starts a seven-second countdown before booting VxWorks. You can interrupt the boot process by pressing any key in order to set the appropriate boot parameters.

You interact with the boot loader shell at a terminal console that is usually established by connecting a serial port of the target to a serial port on the host and starting a terminal application on the host. For information about the setup required to establish communication over the serial port of a particular target, see the reference documentation for the BSP in question.

Project Configuration

While the VxWorks boot application can be used to boot both uniprocessor and SMP images, the VSB project upon which it is based must be set to uniprocessor mode (which is not the default for VSB projects). If it is not set to uniprocessor mode, the boot process will not work properly.

For wrtool, use the -up option. For Workbench, select the uniprocessor option from the Processor Mode drop-down list in the Project Setup dialog.

The VIP project must be created with the PROFILE_BOOTAPP profile, which includes the INCLUDE_BOOT_SHELL component for the boot loader command shell.

The default configuration of the boot application is designed for a networked target, and must be re-configured with your host and target network addresses, the full path and name of the file to be loaded, the user name, and so on. You can interactively change the default parameters using the boot loader shell, or use a static configuration method (by setting the **DEFAULT_BOOT_LINE** configuration parameter).

Related information

VxWorks Boot Application Command Shell on page 32
VxWorks Boot Parameters on page 34
VxWorks Boot Parameters Reference on page 36

2. VxWorks Boot Application Command Shell

The VxWorks boot application command shell allows for interactively changing boot parameters, rebooting VxWorks, managing the boot process, and other operations. Various VIP components provide sets of shell commands.

Command Shell and the Boot Process

When the boot application starts, it prints a banner page. If the boot application is not configured with its command shell, the boot process executes without a countdown, booting the VxWorks image. If the boot application shell is included (with the INCLUDE_BOOT_SHELL component), the boot process executes with a countdown delay. The countdown can be interrupted (by pressing any key) to access the boot application command shell.

The command shell appears with this prompt:

```
[VxWorks Boot]:
```

INCLUDE_BOOT_SHELL Commands

h	Help command. Print a list of available boot commands and flags; the boot device in use, image path, and image file name; and the boot devices available in the boot loader.
?	Same as h.
@	Boot (that is, load and execute the VxWorks image file) using the current boot parameters.
$ [*paramString*]	Boot (that is, load and execute the VxWorks image file). If used without a parameter string, the command is the same as @. The parameter string can be used to set boot loader parameter values all at once, instead of interactively.
p	Print the current boot parameter values.
c	Change the current boot parameter values.
l	Load the VxWorks image using the current boot parameters, but without initiating execution.
g *adrs*	Go to (execute at) the hex address *adrs* .
e	Display a synopsis of the last occurring VxWorks exception.
v	Display the boot application banner page with the BSP name and boot application version number.

WNDRVR

INCLUDE_BOOT_MEM_CMDS Commands

d *adrs* [, *n*]	Display *n* words of memory starting at hex address adrs. If *n* is omitted, the default is 64.
m *adrs*	Modify memory at location adrs (hex). The system prompts for modifications to memory, starting at the specified address. It prints each address, and the current 16-bit value at that address, in turn. You can use any of the following options at the prompt: **ENTER** Pressing the **ENTER** key alone does not change the address specified with *adrs*, but continues prompting at the next address. *number* Entering a number sets the 16-bit contents at the memory address to that number. **.** Entering a **.** character (period) leaves the address unchanged, and quits.
f *adrs*, *nbytes*, *value*	Fill *nbytes* of memory, starting at *adrs* with value.
t *adrs1*, *adrs2*, *nbytes*	Copy *nbytes* of memory, starting at *adrs1*, to *adrs2*.

INCLUDE_BOOT_ETH_MAC_HANDLER Command

M [*dev*] [*unitNo*] [*MAC*]	Set and display Ethernet MAC address. For example: M motfcc0 00:A0:1E:00:10:0A In this case, the device is motfcc, the unit number is zero, and the MAC address is 00A01E00100A.

INCLUDE_BOOT_EDR_SUPPORT Commands

P	Print the error log for the error detection and reporting facility.
C	Clear the error log for the error detection and reporting facility.

INCLUDE_BOOT_FILESYSTEM Commands

devs	Display a list of all devices known to the I/O system. It performs the same function as the kernel shell C interpreter command of the same name.
cd *pathName*	Change directory to the one specified.
pwd	Print the current directory path.

rm *fileName*	Remove the specified file.
ls	List the contents of the current directory.
cp *src dest*	Copy the specified file to the specified destination.

INCLUDE_BOOT_USB_SHOW Command

usbShow	Display the logical unit numbers of the USB device.

INCLUDE_BOOT_UEFI_SUPPORT Commands

bootMpShow	Display the MP/ACPI boot parameters.
uefiPrintMemoryMap	Display the UEFI memory descriptor table.
uefiPrintDebugInfo	Display UEFI debug information.
uefiReset	Execute a UEFI system reset.
uefiSetVirtualMap	Execute the UEFI SetVirtualAddrMap( ) function.

Related information

VxWorks Boot Application on page 31
VxWorks Boot Parameters on page 34
VxWorks Boot Parameters Reference on page 36

3. VxWorks Boot Parameters

The VxWorks boot parameters include all the information needed to locate and load a VxWorks image, other settings used to manage the boot process, and values needed by the VxWorks operating system itself once it has been booted.

As required by the specific boot configuration, the boot parameters can include host and target IP addresses, the full path and name of the VxWorks image to be booted, the user name, and so on. The boot application writes the boot parameters from non-volatile storage into a memory region that is common to both the boot application and the VxWorks image. Once it has been booted, VxWorks obtains the parameter values (such as its IP address) that it requires for the operation of its sub-systems directly from this memory region.

Boot parameters can be changed interactively at runtime from the boot application command shell, or statically configured when the boot application is created. Interactive changes to boot parameters are retained between reboots for most targets (in a non-volatile device or on disk).

Displaying Current Boot Parameters

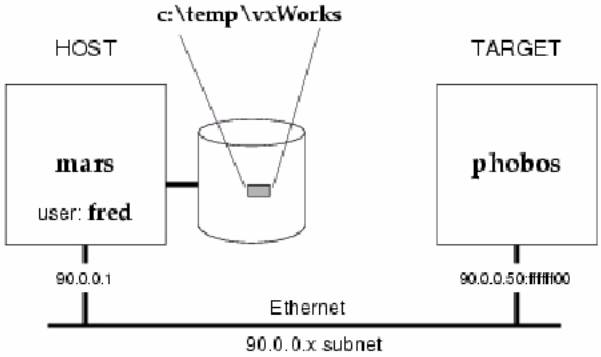
If the boot loader is configured with its command shell, the current set of boot parameters can be displayed interactively with the p command, as follows:

```
[VxWorks Boot]: p
```

A display similar to the following appears. The p command does not actually display unassigned parameters, although this example shows them for completeness.

```
boot device    : ln
unit number            : 0
processor number       : 0
host name              : mars
file name              : c:\tmp\vxWorks
inet on ethernet (e)   : 90.0.0.50:ffffff00
inet on backplane (b)  :
host inet (h)          : 90.0.0.1
gateway inet (g)       :
user (u)               : fred
ftp password (pw)(blank=use rsh) :secret
flags (f)              : 0x0
target name (tn)       : phobos
startup script (s)     :
other (o)              :
```

These configuration settings correspond to the following development environment layout:



Interactive Modification of Boot Parameters

The default boot parameters are set at configuration time with the DEFAULT_BOOT_LINE configuration parameter. They can be changed interactively from the boot application command shell, either individually or as a whole with a string.

To change parameters on an individual basis, first use the c (change) command at the boot prompt, and then enter a new value for each parameter as it is displayed.

If a particular field already has the correct value, simply press **ENTER**. To clear a field, enter a period (.), then press **ENTER**. To go back to change the previous parameter, enter a dash (-), then press **ENTER**. If you want to quit before completing all parameters (but saving your changes), press **CTRL+D**.

To change more than one parameter at time, use the $ command at the command shell prompt with a parameter string. The syntax for the string is as follows:

```
$devName(unitNum,procNum)targetsNameForHost:/filePathh=ipAdre=ipAdrb=ipAdrg=ipAdru=usrName[pw=
passwdf=flagNumtn=targetNames=scriptNameo=other
```

For example:

```
[VxWorks Boot]:$gei(0,0)host:vxWorks h=90.0.0.3 e=90.0.0.50 u=mytarget
```

The first elements are are the values for the boot device, unit number, processor number, host name, and file name parameters. The parameters with assignments (using the **=** operator) use the letter codes shown in parentheses by the p command. The order of the parameters with assignments is not important; and you may omit any that are not relevant.

This method can be particularly useful when booting a target from a host script. The changes made to boot parameters are retained between reboots for most types of targets (in a non-volatile device or on disk).

Static Configuration of Boot Parameters

The default configuration of boot parameters is set with the **DEFAULT_BOOT_LINE** configuration parameter. To modify the default setting, edit the boot line string to which the parameter is set.

Related information

VxWorks Boot Application on page 31
VxWorks Boot Application Command Shell on page 32
VxWorks Boot Parameters Reference on page 36

4. VxWorks Boot Parameters Reference

These boot parameters can be set statically or dynamically with VxWorks boot application.

Boot Parameter Names

In the following table, the names of the boot parameters are given as they appear in the boot shell's display of the parameters and their settings. If the parameters have keywords that must be used in static configuration of the boot parameters, or when a boot line string is used to set all the parameters at once, then the keyword is provided in parenthesis after the name.

Boot Parameter Descriptions

boot device	The type of device from which to boot VxWorks. This must be one of the drivers included in the boot loader (for example, enp for a CMC controller). Due to limited space in boot media, only a few drivers can be included. A list of the drivers included in the boot loader image can be displayed in the boot loader shell with the devs or h command.
unit number	The unit number of the boot device, starting at zero.
processor number	For systems with multiple targets on a backplane, the processor number must be set (and unique) for each target. The backplane master must have the processor number set to zero. For multicore systems, the processor number must be set (and unique) for each CPU. Otherwise, a value of zero is commonly used, but is not required.
host name	The name that the target uses for the host machine. It does not need to be the name used by the host itself. The default value for host name is simply host. To avoid any confusion, change it to something more meaningful for your environment. The host name parameter is not used for a network boot process (using TFTP, FTP, or RSH). Its primary use is for accessing remote file systems over non-NFS network connections from the target after VxWorks has booted. Once VxWorks has

	booted, the host name can be displayed in the device list that is produced by the devs command from the VxWorks kernel shell.
file name	The full path name of the VxWorks image to be booted (for example, **c:\myProj\vxWorks**). This path name is also reported to the host when you start a target server, so that it can locate the host-resident image of VxWorks. The path name is limited to a 160 byte string, including the null terminator. Note: For the files of a remote host to be accessible with RSH or FTP, permissions and user identification must be established on both the remote and local systems.
inet on ethernet (e)	The Internet Protocol (IP) address of a target system Ethernet interface, as well as the subnet mask used for that interface. The address consists of the IP address, in dot decimal format, followed by a colon, followed by the mask in hex format (for example, 90.0.0.50:ffffff00).
inet on backplane (b)	The Internet address of a target system with a backplane interface.
host inet (h)	The Internet address of the host that provides the VxWorks image file (for example 90.0.0.1).
gateway inet (g)	The Internet address of a gateway node for the target if the host is not on the same network as the target.
user (u)	The user ID used to access the host for the purpose of loading the VxWorks image file. The user must have host permission to read the VxWorks image file. On a Windows host, the user specified with this parameter must have FTP access to the host, and the FTP password parameter (see below) must be used to provide the associated password. On a Unix host, the user must have FTP, TFTP, or rsh access. For rsh, the user must be granted access by adding the user ID to the host's **/etc/host.equiv** file, or more typically to the user's **.rhosts** file (**~***userName***/.rhosts**).
ftp password (pw)	For FTP or TFTP access, this field is used for the password for the user identified with the user parameter (see above). For RSH access it should be left blank. Note: If this parameter is not used, the boot application attempts to load the VxWorks image using a protocol based on the UNIX RSH utility, which is not available for Windows hosts.
flags (f)	The flags options are specified as a value that is the sum of the values of selected option bits. For information about the flags, see the next table.
target name (tn)	The name of the target system to be added to the host table.
startup script (s)	If the kernel shell is included in the downloaded image, this parameter allows you to pass to it the path and filename of a startup script to execute after the system boots (which can for example, execute RTP applications). A startup script file can contain only the kernel shell's C interpreter commands. Note: Do not add the kernel shell component (**INCLUDE_SHELL**) to a boot application. It conflicts with the boot loader shell. Adding it causes project configuration errors.
other (o)	This parameter is generally unused, but is available for applications. It can be used, for example, be used to specify the default network interface when booting from a file system device.

flags Boot Parameter Options

SYSFLG_STD_DTB	0x00 0x01	Instructs U-Boot to run either the legacy VxWorks boot branch (0x00) or the standard Linux boot branch (0x01). Also instructs the runtime code on what type DTB to use. This option is board specific; refer to your target documentation.
SYSFLG_DEBUG	0x02	Load local symbols as well as globals. Note: Loading a very large group of symbols can cause delays of up to several minutes while Workbench loads the symbols. In addition, this option can only be used with the downloadable symbol table; it cannot be used with the standalone symbol table.
SYSFLG_NO_AUTOBOOT	0x04	Do not auto-boot.
SYSFLG_QUICK_AUTOBOOT	0x08	Auto-boot fast (short countdown).
SYSFLG_NO_SECURITY	0x20	Disable login security. Do not ask for password for network login.
SYSFLG_AUTOCONFIG	0x40	Use DHCP or bootp to obtain target IP and gateway boot parameters.
SYSFLG_TFTP	0x80	Use tftp to get boot image.
p**SYSFLG_SYS_MODE_DEBUG**	0x400	System is in debug mode.
SYSFLG_AUTOFILE	0x800	Use bootp or DHCP to get boot file.

DHCP Use

The 0x40 option bit (**SYSFLG_AUTOCONFIG**) can be used with the flags parameter so that the following boot parameters are set by the DHCP server:

- the target's IP address and subnet mask, which is specified with the **inet on ethernet** (e) boot parameter
- the gateway router address, which is specified with the **gateway inet** (g) boot parameter

The 0x800 option bit (**SYSFLG_AUTOFILE**) can be used with the flags parameter so that the following boot parameters are set by the DHCP server:

- the target's IP address and subnet mask, which is specified with the **inet on ethernet** (e) boot parameter
- the name of the image file to be booted, which is specified with the **file name** boot parameter
- the IP address of the host that provides the VxWorks image file, which is specified with the **host inet** (h)

If the 0x40 or 0x800 bit are used, and the DHCP server has not been configured to provide the specified parameter values, the boot application uses values previously stored in non-volatile media (if provided by target). If required parameters are incorrect, the target will fail to boot.

Related information

VxWorks Boot Application on page 31
VxWorks Boot Application Command Shell on page 32
VxWorks Boot Parameters on page 34

WNDRVR

11. DEPLOYMENT CONSIDERATIONS

1. MAC Addresses and VxWorks Deployment

If you are going to deploy multiple instances of VxWorks on the same network, you should provide each instance with unique MAC Address.

By default, VxWorks uses the MAC address defined in the DTB file to program the network interface. This is not problem if only there is only one device using it, or for testing purpose. For formal product development and deployment, however, you must implement a method for changing it for multiple instances. These approaches are recommended:

- Use a pre-stage boot loader (for example, U-Boot) to change the MAC address stored in the DTB to a new value. You can do so by using an algorithm to calculate the MAC address based on hardware information, or to retrieve the MAC address from on board non-volatile media.
- Implement code for the function stub endMacGet( ), which is defined in the BSP's **board_func_table{}** structure. If you do so, the network subsystem will use the MAC address that it retrieves instead of the one defined in the DTB.

For further details, refer to the BSP Supplement. To find this, navigate to the General BSP category located on the Wind River product documentation site.

12. SECURELY BOOTING VXWORKS

You can arrange to securely boot Intel, ARM and Power PC devices into the VxWorks operating system.

When a board boots securely, this means every stage of the boot process is checked. VxWorks will only run when the intended, encrypted and signed executable files are loaded at each stage. If unsigned or incorrectly signed boot loader or kernel executables are loaded at any point, then the boot process stops and the board does not run VxWorks.

Examples of VxWorks secure boot are provided in the *VxWorks 7 Security Programmer's Guide: VxWorks Security Overview* for all three supported CPU architectures. These examples use both the UEFI and the U-Boot bootloaders to enforce secure booting of the kernel. Refer to the *VxWorks 7 Security Programmer's Guide: Secure Boot with UEFI* for specific workflow details.

Table 1. VxWorks Secure Boot Devices

CPU Device	Architecture	Boot Loader Used
Intel (e.g. SkyLake and CamelBack Mountain)	Intel (64-bit VxWorks kernel only)	UEFI boot loader
NXP T2080 and P4080	PowerPC	U-Boot boot loader
NXP SabreSD I.MX6 and Xilinx Zynq-7000	ARM	U-Boot boot loader